New Datasets and Methods for `day-in-the-life` Continual Learning Benchmark

Aksel Kretsinger-Walters
adk2164@columbia.edu

Christian Montgomery
cm4521@columbia.edu

Maria Garmonina
mkg2169@columbia.edu

Abstract

1 Introduction and Motivation: MARIA

Continual learning explores how AI systems can adapt and continue training and learning at deployment time. This property is necessary for systems to adapt to real-world scenarios with data distribution shifts and constantly changing information. Models operating in continual learning scenarios inherently struggle with (catastrophic) forgetting, and while there are multiple approaches for lessening or mitigating this phenomenon [1], researchers in the area have not developed a comprehensive, real-world benchmark for continual learning tasks. Most papers we are familiar with in the field operate on MNIST or CIFAR datasets and are able to prove the effectiveness of the respective methods in this very particular setup. Our goal is to test continual learning methods on valuable and interesting tasks that resemble a day in a life of a human (hence the title): finding your way around in an environment, learning new skills, reasoning through hard questions, etc. This manuscript describes our contributions along three dimensions (infrastructure, datasets, and methods) to the larger `day-in-the-life` benchmark. Our main additions and findings are the following:

- For the replay side of Continual Learning, we implemented Dark Experience Replay (DER++) inside the benchmark. In addition, we designed a buffer management policy that allocates buffer capacity based on task similarity rather than uniform reservoir. The intention here was that storing logits along with raw samples, and protecting tasks that are most distinct from the current one, would reduce forgetting more than basic Experience Replay (ER). [Initial four-task results were more nuanced than expected, so a follow-up experiment showed. . . .]
- Sth about DualPrompt
- Sth about new datasets and infrastructure for them

2 Literature Review

Continual learning methods are usually classified into five main areas – they can be based on regularization, replay (also called rehearsal), optimization, representation, and architecture [1]. This classification is not strict, and there is often overlap between the categories. To give some examples of methods from different categories, EWC [2] falls under regularization, GEM [3] is a dual replay- and optimization-based approach, Side-Tuning [4] is a representational method, and Piggyback [5] is an architectural method. The methods we explore and implement for `day-in-the-life` fall under replay (DER [6]) and representation (DualPrompt [7]).

2.1 Experience Replay: CHRISTIAN

Replay is considered one of the simplest and most effective methods to handle catastrophic forgetting. Within replay, basic Experience Replay keeps a small buffer of past examples and trains on a mix of current and replayed samples. Buzzega et al. [6] proposed Dark Experience Replay (DER and DER++), which adds the model's own past output logits to the buffer and uses an MSE distillation loss to keep current predictions close to those stored logits. DER++ further regularizes against the stored hard labels with a second cross-entropy loss term. The intuition is that logits encode richer information than labels alone, so matching them gives stronger preservation of existing knowledge. Buzzega et al. showed DER++ beats ER and several other baselines across CIFAR and Tiny ImageNet variants.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

2.2 Similarity Weighting: CHRISTIAN

A separate question is which examples should fill the replay buffer. Aljundi et al. proposed Maximally Interfered Retrieval (MIR) [8], which concerns selecting optimal examples from the buffer by deciding which stored examples to replay based on whose loss would most increase under the current update. Aljundi et al. also introduced Gradient-based Sample Selection (GSS) [9], which populates the buffer with samples that maximize gradient diversity, on a sample-level rather than task-level. Both MIR and GSS target buffer management, but neither directly addresses how to distribute buffer capacity across tasks.

Roth et al.'s FoMo-in-Flux [10] is one of the closest recent benchmarking studies to this project. They study continual pretraining of multimodal models and look at how to split the training mix between pretraining data, downstream task data, and buffer contents. However, they leave open the question of how the buffer itself should be partitioned across past tasks, which is the gap we tried to fill.

The idea that tasks have distinct geometric structure in embedding space comes from Liang et al.'s Mind the Gap [11], which shows that embedding representations cluster by concept in pretrained models. This is what makes a similarity-aware buffer policy plausible: if one can measure how similar two tasks are by comparing their mean embeddings, one can use that to decide how much buffer space each task requires after the model is trained on that task.

2.3 Prompting in Continual Learning: MARIA

Prompting appeared as a representational approach instructing a pretrained model to adapt to new knowledge in the continual learning setting in L2P [12]. Instead of relying on example memorization and replay, the knowledge about new tasks is injected into the model via training a small pool of parameters (called prompts) that are selected and added to input tokens.

DualPrompt [7] is a method inspired by L2P. It also adds a small set of new parameters (prompts) to train on top of a frozen backbone to instruct the large model to learn sequentially arriving tasks. Typically, prompts constitute 0.2-0.6% of the full model size. In DualPrompt, prompts are explicitly divided into task-specific and task-invariant instructions, which gives a sizable increase in performance as compared to the original L2P framework. All the parameters are trained together with a learnable classifier (the target task in the DualPrompt paper is classification). At test time, the prompt parameters are prepended to the embedding feature extracted from the frozen backbone model in the multi-head attention layers, which is then sent to the rest of the model.

3 Methodology

We are contributing to the `day-in-the-life` benchmark in three directions: methods that we test on the benchmark, the infrastructure of the benchmark's repository, and datasets. The new methods were tested on the `day-in-the-life`

3.1 Backbone Model

We considered multiple base models to benchmark, fine-tune and test our CL algorithms on; these included the Qwen, SmolVLM, and Gemma model families. During our discussions, we landed on a few highest priority characteristics for selecting the default base model: ease of use and access (open-source), size (to increase the accessibility of the benchmark), quality and breadth of documentation (for dependable baseline performance), and community adoption (familiarity, edge-case debugging, guidance). Taking these criteria into account, we arrived at Qwen3-VL-2B-Instruct. This model is well documented and popular within the community, supports vision and text, and small enough to fit on the commonly-accessible Nvidia L4. In addition, its performance on many datasets lies in the "sweet spot": not so strong that the model is saturated, obfuscating the results between different CL algorithms, and not so weak that additional training would yield negligible benefit. Finally, the suite of Qwen3-VL models range in size from 2B to 32B parameters, allowing benchmarkers with greater compute resources to upscale.

3.2 DER Implementation: CHRISTIAN

To implement DER++, we extend the `day-in-the-life` Experience Replay trainer, which also serves as a baseline for determining whether logit distillation actually helps. Our DER++ trainer follows Buzzega et al. [6] closely. The replay buffer is extended so each stored example carries three things: the original input tokens, the labels, and the model's output logits at training time. To keep memory usage reasonable, logits are stored as float16 numpy arrays and reconstructed into padded tensors during collation. Training on a batch combines current-task examples with sampled replay examples and uses a three-term loss:

$$\mathcal{L} = \mathrm{CE}(\text{current}) + \alpha \cdot \mathrm{MSE}(\text{replay logits}) + \beta \cdot \mathrm{CE}(\text{replay labels}) \tag{1}$$

The first term is the standard cross-entropy loss on the current task. The MSE term penalizes divergence between the model's current logits on a replayed example and the logits that were stored when that example was first seen. The second cross-entropy term applies the original hard labels to the same replayed examples. The two coefficients α and β have been tuned as

hyperparameters, resulting in an optimal setting of $\alpha = \beta = 0.5$ to balance weighting equally between the logit-distillation and label-replay loss terms.

3.3 Similarity Weighting Implementation: CHRISTIAN

We also implement two similarity-weighted buffer variants (one each for ER and DER++). The similarity-weighted buffer allocates slots to past tasks in inverse proportion to how similar each one is to the current task, with a small floor so that no task is fully evicted. For each task, we compute a centroid embedding by sampling 32 examples, running them through the pretrained base model, and mean-pooling the last hidden state across non-padding tokens. This gives one vector per task that summarizes where the task lives in representation space. Centroids are computed once before training begins, using the frozen pretrained model, so that the similarity metric does not drift as the model fine-tunes.

When the model finishes task A and is about to start task B, the buffer is rebalanced. For each prior task i in the buffer, the dissimilarity to the upcoming task is computed as:

$$d_i = \max(1 - \text{cos\_sim}(\text{embed}(T_i), \text{embed}(T_{\text{new}})),\ 0.01) \tag{2}$$

We clip the dissimilarity at a small positive value so that even near-identical tasks get nonzero weight in the formula, then normalize across all prior tasks and translate the weights into slot counts. Letting B denote total buffer size, K the number of prior tasks, and floor a per-task minimum (set to 2 in our implementation):

$$n_i = \text{floor} + \text{round}(w_i \cdot (B - \text{floor} \cdot K)) \tag{3}$$

After computing the new allocation, any task that exceeds its quota has the excess randomly evicted, and the new task fills the remaining slots. As an example, suppose the buffer holds 100 slots and there are three prior tasks A, B, C with cosine similarities to the new task D of 0.9, 0.5, and 0.1 respectively. After similarity normalization, Task A receives roughly 7 slots, Task B 33 slots, and Task C 60 slots, since C is the most distinct from D. Meanwhile, knowledge of Task A is largely allowed to drift during training, since training on D will keep Task A's representations refreshed by virtue of their similarity. The same allocation logic is reused across both Sim-ER and Sim-DER++ trainers; the only difference is whether logits are stored alongside the buffer entries.

3.4 DualPrompt Implementation: MARIA

3.5 New Datasets

Due to the variety of datasets, GPUs, and contributors, a standard contribution and benchmarking pipeline had to be formed and agreed to. The per-dataset deliverable consists of

1. A function to download and save the raw data
2. A pre-processing function to convert the data into desired form (i.e. raw tensor -> image)
3. A loader function to load the processed dataset from a predetermined location on disk to memory
4. A configuration dictionary with context, prompting, and post-processing to ensure the LLM's answer format matched the dataset's provided answer format
5. Documentation on the performance of the benchmark LLM out-of-the-box, after 1 epoch of LoRA finetuning, and after 5 epochs of finetuning
6. A hook to couple the configuration dictionary to the loader function
7. A psuedo-deterministic train/eval/test split in the loader, if none is specified by the dataset authors

Following this protocol, we added eight datasets to the benchmark. Three carry native train splits and participate in the continual learning loop; the remaining five are loaded eval-only and are drawn from the Qwen3-VL Technical Report Table 4 [13] so that base-model scores can be cross-referenced against published numbers.

The datasets we added are the following:

- **ChartQA** (Masry et al. 2022) – Visual QA over charts and plots, pre-split into `train / val / test`. The Hub version stores `label` as a plain string and `image` as `Value('binary')` bytes; preprocessing casts the image column to `datasets.Image()` and copies `label` directly to `answer`.
- **ScienceQA** (Lu et al. 2022) – Multiple-choice science QA with accompanying images. Text-only examples are filtered out and the integer `answer` index is resolved to the corresponding choice string.

- **VCR (A-OKVQA stand-in)** – The original VCR dataset (Zellers et al. 2019) requires a signed license and is not redistributable on the Hub. We substitute A-OKVQA (Schwenk et al. 2022), which has the same multiple-choice structure; the correct choice index is resolved to an `answer` string. The test split has `correct_choice_idx=None`, so `answer` is empty for test rows.
- **MMMU-Pro** (Yue et al. 2024b) – Multi-discipline, expert-level multiple-choice VQA covering subjects from art history to circuit analysis. We load the `standard (4 options)` configuration and restrict to single-image rows (the dataset can reference up to seven images per question). The stringified `options` list is parsed and the letter answer is resolved to its option text.
- **LogicVista** (Xiao et al. 2024) – Visual logical-reasoning multiple-choice. The options are embedded in the question text and the reference answer is a single letter, so the model is prompted to emit only `A/B/C/...` and scoring is exact-match on the letter.
- **RoboSpatial-Home** (Song et al. 2025a) – Spatial-understanding QA over indoor scenes, split into three task types: `context` (point at a region), `compatibility` (yes/no), and `configuration` (short phrase). Pointing answers are scored against a per-row binary mask, and the Qwen3-VL JSON suffix is baked into context-row questions at preprocess time so the runner stays task-agnostic.
- **ERQA** (Team et al. 2025) – Embodied-reasoning short-answer VQA, the benchmark used in the Gemini Robotics paper. We load FlagEval's parquet mirror (the original ships as TFRecords) and restrict to single-image rows.
- **HallusionBench** (Guan et al. 2023) – Yes/no diagnostic probes for visual hallucination across charts, OCR, maps, and figures. We use the `image` split (951 rows); native `1/0` labels are mapped to `yes/no` so that exact-match against natural-language model replies works.

3.6 Benchmarking Protocol and Evaluation Metrics

An important part of the dataset delivery pipeline is the benchmarking step. The benefits include data format debugging, validation of the Qwen3 published benchmark score, a mental heuristic for the dataset questions, and a comparison point for the CL algorithm vs a simple LoRA SFT approach.

The first step is matching the experiment setup to the conditions used by the Qwen researchers when conducting the baseline evaluation. In the case that we created our own train/eval/test splits, we concatenate the splits; the Qwen3 benchmarking was done **across the entire dataset**. Additional sources of noise include thinking vs instruct mode, the maximum internal sequence length, and the scoring function (exact match accuracy, relaxed match accuracy, MSE, CE loss, etc)

The second step is drafting an eval config that results in optimal baseline performance. For example, most datasets expect a single-token response. In the absence of additional instruction, the LLM tends to produce a verbose answer, which fails the regex match every time. Providing clear instruction denoises the training signal in backprop, improving performance and interpretability.

The third step is to iteratively run the baseline evaluation until the score matches the published Qwen3 number. This is a sanity check for the dataset and the evaluation pipeline. **Many** bugs were caught and fixed during this step.

The fourth step is to run a simple LoRA SFT on the dataset for one and five epochs. This step places the model on the underperformance <–> saturation spectrum.

The final step is to record the results in the `day_in_the_life/datasets/README.md` table. In addition to the quantitative results included in the table, qualitative observations, shortcuts, compromises, caveats, etc. should be appended to the bottom section.

Benchmark Performance

Dataset	Metric	Model	Approach	Hardware	Score (No-Finetune)	Score (Finetune 1 epoch)	Score (Finetune 5 epochs)	Notes
ChartQA	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.7240	0.7100	0.7120	Not in Qwen3-VL Tech Report Table 4 — see Methodology investigation
ERQA	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.4390	0.4667	0.4667	Paper: 0.413 · ours: 0.439

Figure 1: A snippet of the dataset benchmarking table.

The full table of benchmarking results is included in the Appendix as Figure 2.

3.7 Changes to Processing Existing Datasets: Maria

4 Experimental Results

4.1 Qwen-4b Performance Reproduction

Before testing any CL algorithms, it's necessary to ensure that the baseline performance of the Qwen3-VL-2B-Instruct model on the datasets matches the published numbers. We've outlined many of the hurdles and bugs overcome in this process in Section 3.6, but here we'll focus on some of the gray-area decisions and tradeoffs made.

Inference Settings The Qwen3-VL Tech Report prescribes `temperature=1.0, top_p=1.0, top_k=40, presence_penalty=2.0, max_tokens=32768` for the small Instruct family (2B/4B/8B). The pre-fix default was greedy `temperature=0.0` with `max_tokens=128`. The Instruct model is post-trained on long chain-of-thought data, so it expects to reason before answering — but a 128-token output budget cuts off the reasoning trace and forces the model to commit to an answer prematurely. The fix: a `cot=True` flag in the configuration dictionary that swaps in the paper-spec sampling, and an increased `max_tokens` to allow the model to reason through the question.

Multiple Choice Questions In some cases, chain of thought reasoning resulted in a paraphrasing and corruption of critical answer format instruction. The compromise solution reached was an eval flag `mcq_letter_format` that invoked a minimal postprocessing function to extract the answer and convert it to the correct format.

3D Coordinate Matching There were some hidden bugs in the 3D coordinate prediction, causing massive error and disparity with the published eval. It was necessary to inform the base VLM model as to whether the bounding box targets were normalized to $[0, 1]$, or $[0, 1,000]$ space, or expected in raw pixel location space.

4.2 New Methods' Performance: CHIRSTIAN and MARIA

4.3 Ablations for Replay and Similarity Weighting: CHIRSTIAN

4.4 Learning Rate Ablations for Baseline Training: MARIA

4.5 DualPrompt Ablations: MARIA

5 Further Discussion

5.1 Challenges and Limitations: CHRISTIAN

The DER++ and similarity buffer analysis had the following limitations. Tasks are still learned sequentially without revisiting (once a dataset is visited in the 8-day curriculum, it is not seen again), which is farther from the Day-in-the-Life goal of mimicking how humans learn through spaced rehearsal across topics. The buffer size remains a small fraction of per-task training

data, leaving limited room for similarity-based redistribution to take effect, and the most recent task continues to occupy a disproportionate share of capacity. Task centroids used for similarity weighting are pre-computed from the frozen base model rather than updated as the model fine-tunes, so the similarity metric can drift from what the current model actually represents. Lastly all ablations were also run with a single seed, so variance across runs is not yet quantified to determine the effect of noise.

5.2 Future Directions

There's lots of room for growth for this benchmark. Broadly speaking, that includes api and customization improvements, code hardening and bug squashing, more datasets, more CL algorithms, and more submissions to the leaderboard.

We will focus on the work outstanding for our own team. We plan to implement a PyTorch Lightning training loop to supplement the existing HuggingFace Trainer, PyTorch SFT loop, and tensorflow SFT loop. This would permit pytorch-lightning-based CL algorithms and training solutions.

In addition, we plan to implement a model-merging CL algorithm following the example from "A Practitioner's Guide to Continual Multimodal Pretraining." [10]. This CL algorithm performed very well in the paper's experiments - consistently outperforming the other methods - and is a very different approach to the replay and prompting methods we have implemented so far.

6 Conclusion

6.1 Summary of Findings: ALL

6.2 Contributions: ALL

Contributions of each team member:

- Aksel: Initial integration tests, cleanup, and bug fixes; contribution and evaluation of 8 novel datasets; introduction and implementation of standard dataset onboarding pipeline; scoping and selection of default VLM model. *Future Work* pytorch-lightning training loop implementation; model-merging CL algorithm, following the example from "A Practitioner's Guide to Continual Multimodal Pretraining." [10]
- Christian: Implementation of Dark Experience Replay (DER++) trainer faithful to Buzzega et al. [6]; design and implementation of a similarity-weighted buffer management policy that allocates buffer capacity to prior tasks in inverse proportion to their cosine similarity to the current task in pretrained embedding space, applied to both ER and DER++ to produce Sim-ER and Sim-DER++ trainers; initial five-way ablation comparing No Replay, Uniform ER, Uniform DER++, Sim-ER, and Sim-DER++ on a four-task subset of the curriculum. *Future Work:* hyperparameter sweeps on buffer size and per-task recency cap; dynamic recomputation of task centroids during training rather than freezing them at the start; multi-seed evaluation on the full 8-dataset interleaved curriculum.
- Maria:

Acknowledgments

We are grateful to Prof. Richard Zemel, Thomas Zollo, and Amogh Inamdar for their mentorship and advice in this project.

Code Availability

All the code used in this project is available at `https://github.com/AmoghSInamdar/day-in-the-life`.

References

[1] Wang, Liyuan, Xingxing Zhang, Hang Su, and Jun Zhu. "A comprehensive survey of continual learning: Theory, method and application." IEEE transactions on pattern analysis and machine intelligence 46, no. 8 (2024): 5362-5383.

[2] Kirkpatrick, James, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan et al. "Overcoming catastrophic forgetting in neural networks." Proceedings of the national academy of sciences 114, no. 13 (2017): 3521-3526.

[3] Lopez-Paz, David, and Marc'Aurelio Ranzato. "Gradient episodic memory for continual learning." Advances in neural information processing systems 30 (2017).

[4] Zhang, Jeffrey O., Alexander Sax, Amir Zamir, Leonidas Guibas, and Jitendra Malik. "Side-tuning: a baseline for network adaptation via additive side networks." In European conference on computer vision, pp. 698-714. Cham: Springer International Publishing, 2020.

[5] Mallya, Arun, Dillon Davis, and Svetlana Lazebnik. "Piggyback: Adapting a single network to multiple tasks by learning to mask weights." In Proceedings of the European conference on computer vision (ECCV), pp. 67-82. 2018.

[6] Buzzega, Pietro, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. "Dark experience for general continual learning: a strong, simple baseline." Advances in neural information processing systems 33 (2020): 15920-15930.

[7] Wang, Zifeng, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren et al. "Dualprompt: Complementary prompting for rehearsal-free continual learning." In European conference on computer vision, pp. 631-648. Cham: Springer Nature Switzerland, 2022.

[8] Aljundi, Rahaf, Eugene Belilovsky, Tinne Tuytelaars, Laurent Charlin, Massimo Caccia, Min Lin, and Lucas Page-Caccia. "Online continual learning with maximal interfered retrieval." Advances in neural information processing systems 32 (2019).

[9] Aljundi, Rahaf, Min Lin, Baptiste Goujaud, and Yoshua Bengio. "Gradient based sample selection for online continual learning." Advances in neural information processing systems 32 (2019).

[10] Roth, Karsten, Vishaal Udandarao, Sebastian Dziadzio, Ameya Prabhu, Mehdi Cherti, Oriol Vinyals, Olivier Hénaff, Samuel Albanie, Matthias Bethge, and Zeynep Akata. "A Practitioner's Guide to Continual Multimodal Pretraining." arXiv preprint arXiv:2408.14471 (2024).

[11] Liang, Victor Weixin, Yuhui Zhang, Yongchan Kwon, Serena Yeung, and James Y. Zou. "Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning." Advances in Neural Information Processing Systems 35 (2022): 17612-17625.

[12] Wang, Zifeng, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. "Learning to prompt for continual learning." In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 139-149. 2022.

[13] Bai, Shuai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng et al. "Qwen3-vl technical report." arXiv preprint arXiv:2511.21631 (2025).

Appendix

Benchmark Performance

Dataset	Metric	Model	Approach	Hardware	Score (No-Finetune)	Score (Finetune 1 epoch)	Score (Finetune 5 epochs)	Notes
ChartQA	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.7240	0.7100	0.7120	Not in Qwen3-VL Tech Report Table 4 — see Methodology investigation
ScienceQA	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.9000	0.8680	0.8780	Not in Qwen3-VL Tech Report Table 4 — see Methodology investigation
VCR	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.8480	0.8580	0.8500	VCR proper is licensed; A-OKVQA stand-in (no paper comparison)
MMMU-Pro	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.5660	0.5687	0.5687	Paper: 0.532 · pre-CoT: 0.478 · CoT: 0.566
LogicVista	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.5893	0.5000	0.4565	Paper: 0.532 · pre-CoT: 0.397 · CoT: 0.589
ERQA	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.4390	0.4667	0.4667	Paper: 0.413 · ours: 0.439
RoboSpatial-Home	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.6446	0.7040	0.7296	Paper: 0.617 · ours (after metric fix): 0.645
HallusionBench	Accuracy	Qwen/Qwen3-VL-4B-Instruct	QLoRA (4-bit base, LoRA r=16); 2K train / 500 test subsample; vLLM eval	1x NVIDIA L4 (24 GB VRAM), 125 GB RAM, 32 vCPU	0.6751	0.5714	0.5714	Paper: 0.576 · ours: 0.675

Figure 2: The full benchmark result table.